

Implementing Software Contracts with Effect Handler

Language PCF

$$\tau ::= o \\ | \tau \rightarrow \tau$$

$$o ::= \text{num} \\ | \text{bool}$$

$$e ::= 0 \mid -1 \mid 1 \mid \dots \mid \text{tt} \mid \text{ff} \mid x \\ | (\lambda x : \tau. e) \mid (\mu x : \tau. e) \\ | e + e \mid e - e \mid e \wedge e \mid e \vee e \\ | e e \\ | \text{if } e \text{ then } e \text{ else } e \\ | \text{zero?}(E)$$

$$v ::= 0 \mid -1 \mid 1 \mid \dots \mid \text{tt} \mid \text{ff} \mid x \\ | (\lambda x. e) \mid (\mu x. e)$$

$$E ::= [] \\ | E + e \mid v + E \mid E \wedge e \mid v \wedge E \mid E \vee e \mid v \vee E \\ | E e \mid v E \\ | \text{if } E \text{ then } e \text{ else } e \\ | \text{zero?}(E)$$

$$E[\lambda x. e \ v] \mapsto E[\{v/x\}e]$$

$$E[\text{if tt then } e_1 \text{ else } e_2] \mapsto E[e_1]$$

$$E[\text{if ff then } e_1 \text{ else } e_2] \mapsto E[e_2]$$

Language Contract extends PCF with Contracts

Surface syntax:

$$\tau ::= \dots \\ | \text{con}(\tau)$$

$$e ::= \dots \\ | \text{mon}_j^{k,l}(\kappa, e)$$

$$\kappa ::= \text{flat}(e) \mid \kappa \rightarrow \kappa \mid \kappa \stackrel{d}{\mapsto} (\lambda x : \tau. \kappa)$$

$$E ::= \dots \\ | \text{mon}_l^{l,l}(\kappa, E)$$

Evaluation terms is added with an error:

$$v ::= \dots \\ | \text{error}_p$$

$$e ::= \dots \\ | \text{error}_p$$

$$E[\text{mon}_j^{k,l}(\text{flat}(e), v)] \mapsto E[\text{if } e \ v \text{ then } v \text{ else blame}(k)]$$

$$\begin{aligned}
E[\text{mon}_j^{k,l}(\kappa_1 \rightarrow \kappa_2, v)] &\mapsto E[\lambda x. \text{mon}_j^{k,l}(\kappa_2, v \text{mon}_j^{l,k}(\kappa_1, x))] \\
E[\text{mon}_j^{k,l}(\kappa_1 \xrightarrow{d} (\lambda x. \kappa_2), v)] &\mapsto E[\lambda x. \text{mon}_j^{k,l}(\{\text{mon}_j^{l,j}(\kappa_1, x)/x\} \kappa_2, v \text{mon}_j^{l,k}(\kappa_1, x))] \\
E[{}^l\text{error}_p] &\mapsto {}^l\text{error}_p
\end{aligned}$$

We use the monitor construct $\text{mon}_l^{+,-}(\kappa, e)$, and blame positive for flat $+$, with labels l is the location of the contract. This will make sure that *indy* reduction rule works for dependent contracts.

We have not added effectful contracts to this construction. It should be easy, by adding a parameter with monitor about being a contract for effects, or contract for values.

Language Effect extends PCF with Effects

Types are problematic, we may need to return polymorphic types because different effects fuse together into one type.

$$\begin{aligned}
\tau &::= \dots \\
v &::= \dots \\
&\quad | \text{op} && \text{Operation, aka Effect name} \\
e &::= \dots \\
&\quad | v(e) && \text{Calling an effect with parameter} \\
&\quad | \text{handle}\{h\}(e) \\
&\quad | \text{blame}(p) && \text{Blaming party } p \\
\text{Handler } h &::= \text{return } x \rightarrow e \\
&\quad | \text{op}(x) \rightarrow e; h && \text{op} \notin h \\
E &::= \dots \\
&\quad | E(e) \mid v(E) \\
&\quad | \text{handle}\{h\}(E) \\
X_{\text{op}} &::= [] \\
&\quad | X_{\text{op}} + e \mid v + X_{\text{op}} \mid X_{\text{op}} \wedge e \mid v \wedge X_{\text{op}} \\
&\quad | X_{\text{op}} \vee e \mid v \vee X_{\text{op}} \\
&\quad | X_{\text{op}} e \mid v X_{\text{op}} \\
&\quad | \text{if } X_{\text{op}} \text{ then } e \text{ else } e \\
&\quad | \text{zero?}(X_{\text{op}}) \\
&\quad | X_{\text{op}}(e) \mid v(X_{\text{op}}) \\
&\quad | \text{handle}\{h\}(X_{\text{op}}) && \text{when } \text{op} \notin h
\end{aligned}$$

Evaluation terms:

$$\begin{aligned}
v &::= \dots \\
&\quad | \text{blame}(p)
\end{aligned}$$

$$E[\text{blame}(p)] \mapsto \text{blame}(p)$$

Reduction rules in [1] works for both E and X_{op} .

$$\text{handle}\{h\}(v) \mapsto \{x/v\}e \text{ where } \text{return } x \rightarrow e \in h$$

$$\text{handle}\{h\}(X_{\text{op}}[\text{op}(v)]) \mapsto \{v/x\}\{\lambda y. \text{handle}\{h\}(X_{\text{op}}[y]) / \text{resume}\}e \text{ where } \text{op}(x) \rightarrow e \in h$$

This reduction rule is the deep handler, it re-register the handler in the resume context.

[1] also defines a construct for effects with state by using a parameterized variable

$\text{handle}\{h\}\langle p = v_p \rangle(e)$ and its reduction rule:

$$\text{handle}\{h\}\langle p = v_p \rangle(\mathcal{X}_{\text{op}}[\text{op}(v)]) \mapsto \{v/x\}\{\lambda q. \lambda y. \text{handle}\{h\}\langle p = q \rangle(\mathcal{X}_{\text{op}}[y]) / \text{resume}\}\{v_p/p\}e$$

Resume function now receives an extra argument to update the state. This would come in handy for trace contracts, where a “collector” is used similarly to a state. However, the state is only inside the continuation, which will not work with our setup because we only run op once. But if the state is a global or to be precise, enclosed in a closure then it would work (?).

Compiler

Let ϕ a function from Contract to Effect:

Two strategy for this, one is that effect returns a no-type (). Second is effect returns the type of value. In our implementation we let effects return a no-type, but it can be adjusted. In this formal settings, we let effects returns their value types. This makes these equivalent $x = \text{op}_x(x)$.

Technically, the resume can be removed, the expression returns the value right away. However, resume will be used for state, which will come in handy for trace contracts.

Labels are defined correctly as well. We convert the monitoring labels, and apply the compiler recursively, so it should give us the correct labels.

The compiler tries to unwrap the monitor constructs recursively, in doing so, it sets the correct blame labels to be used. This works because the contract construction is explicit. For function contracts (and also dependent contracts), the compiler chooses a new variable and wraps the body into a new function. This aligns with the OCaml implementation.

We use “let” here to define new variables as replacement for complex expressions.

$$\begin{aligned} \phi(\text{mon}_j^{k,l}(\text{flat}(e_1), e_2)) &= \text{handle}\{h\}(\text{op}_x(\phi(e_2))) \\ \text{where } h &= \{\text{op}_x(x) \rightarrow \text{if } \phi(e_1) \text{ } x \text{ then resume } x \text{ else blame}(k); \\ &\quad \text{return } x \rightarrow x\} \end{aligned}$$

$$\phi(\text{mon}_j^{k,l}(\kappa_1 \rightarrow \kappa_2, f)) = \lambda x. \text{let } x' = \phi(\text{mon}_j^{k,l}(\kappa_1, x)) \text{ in } \phi(\text{mon}_j^{l,k}(\kappa_2, \phi(f) x'))$$

$$\begin{aligned} \phi(\text{mon}_j^{k,l}(\kappa_1 \xrightarrow{d} (\lambda x. \kappa_2), f)) &= \lambda y. \quad \text{let } y_1 = \phi(\text{mon}_j^{l,k}(\kappa_1, y)) \text{ in} \\ &\quad \text{let } y_2 = \phi(\text{mon}_j^{l,j}(\kappa_1, y)) \text{ in} \\ &\quad \phi(\text{mon}_j^{k,l}(\{y_2/x\}\kappa_2, \phi(f) y_1)) \end{aligned}$$

$$\phi(\lambda x : \tau. e) = \lambda x : \tau. \phi(e)$$

$$\phi(\mu x : \tau. e) = \mu x : \tau. \phi(e)$$

$$\phi(x) = x$$

$$\phi(e_1 \text{ } e_2) = \phi(e_1) \phi(e_2)$$

$$\phi(e_1 \text{ op } e_2) = \phi(e_1) \text{ op } \phi(e_2), \text{ where op } \in (+, -, \wedge, \vee)$$

$$\phi(\text{if } e_1 \text{ then } e_2 \text{ else } e_3) = \text{if } \phi(e_1) \text{ then } \phi(e_2) \text{ else } \phi(e_3)$$

$$\phi(\text{zero?}(e)) = \text{zero?}(\phi(e))$$

In [2], the authors showed a compiler from trace contracts to normal contracts. We can reuse that construction here as well to adapt the trace contract model.

This makes sense?

Yes. Let's look at some examples taken from Contracts as Pairs of Projections paper. We define two parties s the server, and c the client. They exist to demonstrate the blame is consistent after compilation. We also use l as the location of the contract.

1. $\text{mon}_l^{s,c}(\text{flat}(\text{odd}) \longrightarrow \text{flat}(\text{odd}), \lambda y.y) 2$

Compiles into

$$\lambda x. \text{handle}\{h_1\}(\text{op}_f(\lambda y.y \text{handle}\{h_2\}(\text{op}_v(x)))) 2$$

$$\text{where } h_1 = \{\text{op}_f(x) \rightarrow \text{if odd } x \text{ then resume } x \text{ else blame}(s); \text{return } x \rightarrow x\}$$

$$h_2 = \{\text{op}_v(x) \rightarrow \text{if odd } x \text{ then resume } x \text{ else blame}(c); \text{return } x \rightarrow x\}$$

Original reduction:

$$\text{mon}_l^{s,c}(\text{flat}(\text{odd}) \longrightarrow \text{flat}(\text{odd}), \lambda y.y) 2$$

$$\mapsto \lambda x. \text{mon}_l^{s,c}(\text{flat}(\text{odd}), \lambda y.y \text{mon}_l^{c,s}(\text{flat}(\text{odd}), x)) 2$$

$$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{odd}), \lambda y.y \text{mon}_l^{c,s}(\text{flat}(\text{odd}), 2))$$

$$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{odd}), \text{mon}_l^{c,s}(\text{flat}(\text{odd}), 2))$$

$$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{odd}), \text{if flat}(\text{odd}) 2 \text{ then } 2 \text{ else blame}(c))$$

$$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{odd}), \text{blame}(c))$$

$$\mapsto \text{blame}(c)$$

Compiled reduction:

$$\lambda x. \text{handle}\{h_1\}(\text{op}_f(\lambda y.y \text{handle}\{h_2\}(\text{op}_v(x)))) 2$$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(\lambda y.y \text{handle}\{h_2\}(\text{op}_v(2))))$$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(\lambda y.y \text{handle}\{h_2\}(\text{op}_v(2))))$$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(\lambda y.y \text{handle}\{h_2\}(\text{op}_v(2))))$$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(\lambda y.y (\text{if odd } 2 \text{ then resume } 2 \text{ else blame}(c))))$$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(\lambda y.y \text{blame}(c))))$$

$$\mapsto \text{blame}(c)$$

2. $\text{mon}_l^{s,c}((\text{flat}(\text{odd}) \longrightarrow \text{flat}(\text{odd})) \longrightarrow \text{flat}(\text{even}), \lambda f.f \ 1) \ \lambda x.x + 2$

The monitored term compiled into

$\lambda g. \text{handle}\{h_1\}(op_f((\lambda f.f \ 1) (\lambda y. \text{handle}\{h_2\}(op_g(g \text{handle}\{h_3\}(op_y(y)))))))$

where $h_1 = \{op_f(x) \rightarrow \text{if even } x \text{ then resume } x \text{ else blame}(s); \text{return } x \rightarrow x\}$
 $h_2 = \{op_g(x) \rightarrow \text{if odd } x \text{ then resume } x \text{ else blame}(c); \text{return } x \rightarrow x\}$
 $h_3 = \{op_y(x) \rightarrow \text{if odd } x \text{ then resume } x \text{ else blame}(s); \text{return } x \rightarrow x\}$

Original reduction:

$\text{mon}_l^{s,c}((\text{flat}(\text{odd}) \longrightarrow \text{flat}(\text{odd})) \longrightarrow \text{flat}(\text{even}), \lambda f.f \ 1) (\lambda x.x + 2)$

$\mapsto \lambda g. \text{mon}_l^{s,c}(\text{flat}(\text{even}), (\lambda f.f \ 1) \text{mon}_l^{c,s}(\text{flat}(\text{odd}) \longrightarrow \text{flat}(\text{odd}), g)) (\lambda x.x + 2)$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), (\lambda f.f \ 1) \text{mon}_l^{c,s}(\text{flat}(\text{odd}) \longrightarrow \text{flat}(\text{odd}), \lambda x.x + 2))$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), (\lambda f.f \ 1) \lambda y. \text{mon}_l^{c,s}(\text{flat}(\text{odd}), \text{mon}_l^{s,c}(\text{flat}(\text{odd}), (\lambda x.x + 2) y)))$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), \lambda y. \text{mon}_l^{c,s}(\text{flat}(\text{odd}), \text{mon}_l^{s,c}(\text{flat}(\text{odd}), (\lambda x.x + 2) y)) \ 1)$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), \text{mon}_l^{c,s}(\text{flat}(\text{odd}), \text{mon}_l^{s,c}(\text{flat}(\text{odd}), (\lambda x.x + 2) \ 1)))$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), \text{mon}_l^{c,s}(\text{flat}(\text{odd}), \text{mon}_l^{s,c}(\text{flat}(\text{odd}), 3)))$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), \text{mon}_l^{c,s}(\text{flat}(\text{odd}), \text{if odd } 3 \text{ then } 3 \text{ else blame}(s)))$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), \text{mon}_l^{c,s}(\text{flat}(\text{odd}), 3))$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), \text{if odd } 3 \text{ then } 3 \text{ else blame}(c))$

$\mapsto \text{mon}_l^{s,c}(\text{flat}(\text{even}), 3)$

$\mapsto \text{if even } 3 \text{ then } 3 \text{ else blame}(s)$

$\mapsto \text{blame}(s)$

Compiled reduction:

$\lambda g. \text{handle}\{h_1\}(op_f((\lambda f.f \ 1) (\lambda y. \text{handle}\{h_2\}(op_g(g \text{handle}\{h_3\}(op_y(y))))))) (\lambda x.x + 2)$

$\mapsto \text{handle}\{h_1\}(op_f((\lambda f.f \ 1) (\lambda y. \text{handle}\{h_2\}(op_g((\lambda x.x + 2) \text{handle}\{h_3\}(op_y(y)))))))$

$\mapsto \text{handle}\{h_1\}(op_f((\lambda y. \text{handle}\{h_2\}(op_g((\lambda x.x + 2) \text{handle}\{h_3\}(op_y(y)))))) \ 1))$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(\text{handle}\{h_2\}(\text{op}_g((\lambda x.x + 2) \text{handle}\{h_3\}(\text{op}_y(1))))))$$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(\text{handle}\{h_2\}(\text{op}_g((\lambda x.x + 2) 1))))$$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(\text{handle}\{h_2\}(\text{op}_g(3))))$$

$$\mapsto \text{handle}\{h_1\}(\text{op}_f(3))$$

$$\mapsto \text{if even } 3 \text{ then resume } 3 \text{ else blame}(s)$$

$$\mapsto \text{blame}(s)$$

$$3. \text{mon}_l^{s,c}(\kappa_1 \longrightarrow (\kappa_2 \rightarrow \kappa_3), \lambda x.\lambda y.x + y) 1$$

Where $\kappa_1, \kappa_2, \kappa_3$ are flat contracts. Let κ_1 passes for 1.

Compiles into:

$$\lambda z.\lambda w. \text{handle}\{h_1\}(\text{op}_f(((\lambda x.\lambda y.x + y) (\text{handle}\{h_3\}(\text{op}_z(z)))) \text{handle}\{h_2\}(\text{op}_w(w))))1$$

where $h_1 = \{\text{op}_f(x) \rightarrow \text{if } \kappa_3 \text{ } x \text{ then resume } x \text{ else blame}(s); \text{return } x \rightarrow x\}$

$h_2 = \{\text{op}_w(x) \rightarrow \text{if } \kappa_2 \text{ } x \text{ then resume } x \text{ else blame}(c); \text{return } x \rightarrow x\}$

$h_3 = \{\text{op}_z(x) \rightarrow \text{if } \kappa_1 \text{ } x \text{ then resume } x \text{ else blame}(c); \text{return } x \rightarrow x\}$

Original reduction:

$$\text{mon}_l^{s,c}(\kappa_1 \longrightarrow (\kappa_2 \rightarrow \kappa_3), \lambda x.\lambda y.x + y) 1$$

$$\mapsto \lambda z.\text{mon}_l^{s,c}(\kappa_2 \rightarrow \kappa_3, (\lambda x.\lambda y.x + y) \text{mon}_l^{c,s}(\kappa_1, z)) 1$$

$$\mapsto \text{mon}_l^{s,c}(\kappa_2 \rightarrow \kappa_3, (\lambda x.\lambda y.x + y) \text{mon}_l^{c,s}(\kappa_1, 1))$$

$$\mapsto \text{mon}_l^{s,c}(\kappa_2 \rightarrow \kappa_3, (\lambda x.\lambda y.x + y) 1)$$

$$\mapsto \text{mon}_l^{s,c}(\kappa_2 \rightarrow \kappa_3, \lambda y.1 + y)$$

$$\mapsto \lambda z.\text{mon}_l^{s,c}(\kappa_3, \lambda y.1 + y \text{mon}_l^{c,s}(\kappa_2, z))$$

$$\mapsto \text{Compiled to Effect:}$$

$$\lambda z. \text{handle}\{h_A\}(\text{op}_A((\lambda y.1 + y) \text{handle}\{h_B\}(\text{op}_B(z))))$$

$$\mapsto \text{where } h_A = \{\text{op}_A(x) \rightarrow \text{if } \kappa_3 \text{ } x \text{ then resume } x \text{ else blame}(s); \text{return } x \rightarrow x\}$$

$$h_B = \{\text{op}_B(x) \rightarrow \text{if } \kappa_2 \text{ } x \text{ then resume } x \text{ else blame}(c); \text{return } x \rightarrow x\}$$

Compiled reduction:

$$\lambda z.\lambda w. \text{handle}\{h_1\}(\text{op}_f(((\lambda x.\lambda y.x + y) (\text{handle}\{h_3\}(\text{op}_z(z)))) \text{handle}\{h_2\}(\text{op}_w(w))))1$$

$\mapsto \lambda w. \text{handle}\{h_1\}(op_f(((\lambda x. \lambda y. x + y) (\text{handle}\{h_3\}(op_z(1)))) \text{handle}\{h_2\}(op_w(w))))$

$\mapsto$ May need to reduce the function body?

$\mapsto \lambda w. \text{handle}\{h_1\}(op_f((\lambda y. \text{handle}\{h_3\}(op_z(1)) + y) \text{handle}\{h_2\}(op_w(w))))$

$\mapsto \lambda w. \text{handle}\{h_1\}(op_f((\lambda y. 1 + y) \text{handle}\{h_2\}(op_w(w))))$

These function are equivalent iff $h_1 = h_A$ and $h_2 = h_B$ (needs definition of equivalence). Variable names are normalized $w = z$.

$\lambda w. \text{handle}\{h_1\}(op_f((\lambda y. 1 + y) \text{handle}\{h_2\}(op_w(w))))$

where $h_1 = \{op_f(x) \rightarrow \text{if } \kappa_3 \ x \text{ then resume } x \text{ else blame}(s); \text{return } x \rightarrow x\}$

$h_2 = \{op_w(x) \rightarrow \text{if } \kappa_2 \ x \text{ then resume } x \text{ else blame}(c); \text{return } x \rightarrow x\}$

$\lambda z. \text{handle}\{h_A\}(op_A((\lambda y. 1 + y) \text{handle}\{h_B\}(op_B(z))))$

where $h_A = \{op_A(x) \rightarrow \text{if } \kappa_3 \ x \text{ then resume } x \text{ else blame}(s); \text{return } x \rightarrow x\}$

$h_B = \{op_B(x) \rightarrow \text{if } \kappa_2 \ x \text{ then resume } x \text{ else blame}(c); \text{return } x \rightarrow x\}$

Proof?

1. Maintain the same semantic behavior

Given $C \in \text{Contract}$,

$C \Downarrow v, v \in \text{num}$ Then $\phi(C) \Downarrow v$

$C \Downarrow v, v \in \text{bool}$ Then $\phi(C) \Downarrow v$

$C \Downarrow \text{!error}_p$ Then $\phi(C) \Downarrow \text{blame}(p)$

$C \Downarrow \lambda x. e$ This doesn't work $\phi(C) \Downarrow \lambda y. \phi(e)$. See example 3 above.

Maybe we should use observable behavior for this proof?

2. The compiler produces well-typed Effect program from well-typed Contract program

Should be imminent from the compilation.

Ocaml Implementation

```
1  let OCaml  
2    [@contract : contract_for_x -> contract_for_y -> contract_for_z ->  
   contract_for_f]  
3    [@effect : effect_name * effect_predicate]  
4    f x y z = x + y + z  
5  
6  let f x y z =  
7    let [@contract : contract_for_x] x = x in  
8    let [@contract : contract_for_y] y = y in  
9    let [@contract : contract_for_z] z = z in  
10  
11   (* return are wrapped, for effectful contracts,  
12     we must capture the effects too  
13  
14     the compiler should wrap this with another handler for the return value  
15   *)  
16  
17   (* the normal body wrapped with effects  
18   let [@contract_for_f] ret = x + y + z in  
19   ret  
20   *)  
21  
22   let [@contract_for_f] ret = try_with  
23     (fun () -> x + y + z) ()  
24     {  
25       effc = fun (type e) (eff : e Effect.t) ->  
26         match eff with  
27         | effect_name _ -> Some (fun k ->  
28           let result = (Effect.perform eff);  
29           (* also install handler if this also needs to call effect? *)  
30           let valid = effect_predicate result  
31           if valid  
32             then continue k ()  
33             else discontinue k (Blame ?)  
34         )  
35         | _ -> None (* or blame calling unallowed effects *)  
36     }  
37   in  
38  
39   ret
```

Bibliography

- [1] D. Leijen, “Algebraic Effects for Functional Programming,” technical report MSR-TR-2016-29, Aug. 2016. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/algebraic-effects-for-functional-programming/>

- [2] C. MOY and M. FELLEISEN, “Trace contracts,” *Journal of Functional Programming*, vol. 33, p. e14, 2023, doi: 10.1017/S0956796823000096.
- [3] C. Moy, C. Dimoulas, and M. Felleisen, “Effectful Software Contracts,” *Proc. ACM Program. Lang.*, vol. 8, no. POPL, Jan. 2024, doi: 10.1145/3632930.
- [4] C. Dimoulas and M. Felleisen, “On contract satisfaction in a higher-order world,” *ACM Trans. Program. Lang. Syst.*, vol. 33, no. 5, Nov. 2011, doi: 10.1145/2039346.2039348.